PROPOSAL · NOVEMBER 2026

Large AI models on the laptops we already have

Running 100B-class open models from SSD on 8 to 16 GB machines, as a private coding assistant inside the IDE.

What others have proven, what stands in the way, and a three-step plan with a one-day first step.

THE PROBLEM

The strongest open models are several times larger than a laptop's memory

65 GB

Size of gpt-oss-120b, a leading open 120B-parameter model. Its makers size it for an 80 GB server GPU.

16 GB

RAM in a typical developer laptop. About 10 GB is left for a model once the operating system and IDE are running.

1 TB

SSD in the same laptop. An M4 MacBook Air reads from it at 2.9 GB every second.

TODAY'S TWO CHOICES

- **Small local model.** Fits in RAM and is fast, but weaker on hard problems.
- **Cloud model.** Strong, but source code leaves the laptop and every request is billed.

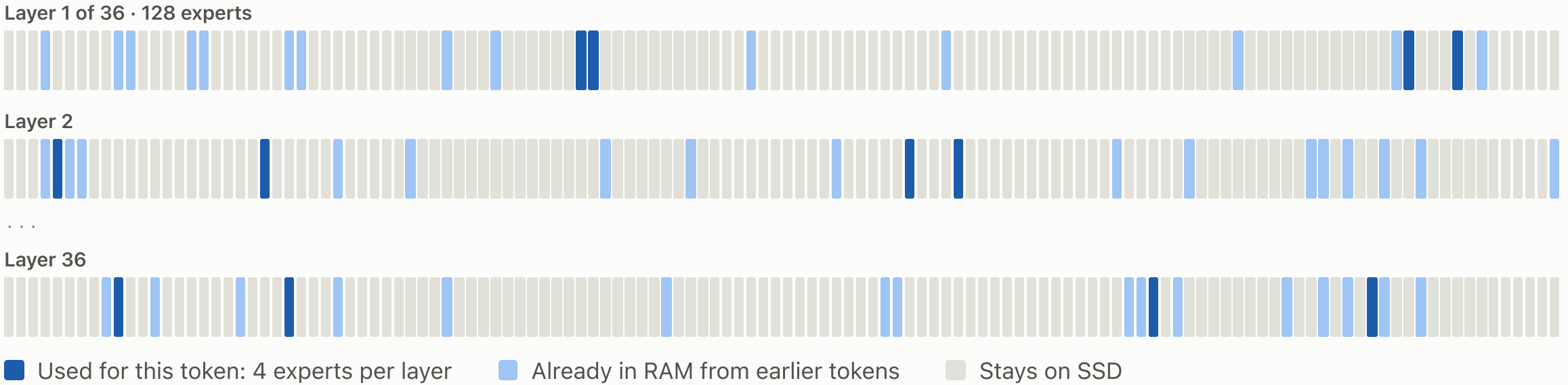
THE OPENING

- Keep the big model on the SSD and bring into RAM only the part each step needs.
- No new hardware, no cloud service, no licence fees: the models are Apache 2.0 or MIT.

Sources: OpenAI gpt-oss-120b model card; Gigazine M4 MacBook Air benchmark; ModelFit 16 GB guide (free-RAM estimate).

THE IDEA

The model stays on SSD, and each word it writes needs only about 3% of it



WHY IT WORKS

- Mixture-of-experts (MoE) models split their knowledge into experts. For each token the model picks a few and ignores the rest.
- gpt-oss-120b holds 117B parameters on disk and uses 5.1B at a time.

WHERE IT DOES NOT WORK

- Dense models, which is most 7B to 70B models, use every weight for every token.
- Run from disk, a dense 70B model takes about 10 seconds per token.

Sources: gpt-oss-120b config (36 layers, 128 experts, 4 per token) and model card; prima.cpp paper, llama.cpp baseline for a dense 70B model. Cell positions are illustrative.

REFINING THE IDEA

“Load only what the task needs” works at three levels

LEVEL 1 · PER TOKEN

The model picks, we fetch

The model's own router chooses the experts for each token. Our daemon reads exactly those from SSD and nothing else.

Evidence. Flash-MoE reads 4 experts per layer on demand and runs a 397B model at 4.4 tokens per second.

LEVEL 2 · PER SESSION

Keep what is hot

Experts used recently and often stay in RAM, so most lookups never touch the SSD.

Evidence. With 15 to 20% of experts held in memory, published systems avoid 50 to 80% of SSD reads.

LEVEL 3 · PER TASK

A profile for coding

Sample real coding prompts once, then pin the experts they rely on. Experts that coding never uses can be left out entirely.

Evidence. Pinning from real text lifts cache hits from 6% to 42%. Removing half the experts cost nothing measurable on a code benchmark.

What does not work: an outside agent choosing weights from the prompt text. The choice is made inside the model, layer by layer, as it runs. Attempts to predict it ahead of time made things slower, by 18% in one project.

Sources: Flash-MoE README; SSD-LLaMA paper; SGLang Expert Pack post; llama.cpp PR #25294 and discussions #23324, #27149; “Half the Experts, All the Code” (arXiv 2607.16721).

PROOF IT WORKS

Five independent teams made this work in 2026

WHO	WHAT THEY RAN	ON WHAT	RESULT
Flash-MoE open-source project, March	Qwen3.5-397B, 209 GB on disk	MacBook Pro M3 Max, 48 GB RAM	**4.4 tokens/s**
SSD-LLaMA research paper, September	Kimi-K2.7-Code, 1 trillion parameters, 544 GiB	Desktop PC, 32 GB RAM, RTX 5090	**1.03 tokens/s**
SGLang Expert Pack LMSYS, August	DeepSeek-V4-Flash, 155 GB	Desktop PC, 32 GB RAM, RTX 5090	**up to 2.0 tokens/s** from 0.29 with Ollama
llama.cpp PR #25294 open pull request, July	GLM-5.2, 254 GB	128 GB workstation	**2.1 tokens/s** 2.4x faster than memory-mapping
llama.cpp PoC #23324 community proof of concept, May	Qwen3-30B-A3B, a 30B model	MacBook M1 Pro, 16 GB RAM	**about 13 tokens/s**
For contrast: stock llama.cpp	Dense 70B model	Desktop, about 10 GB free RAM	about 0.1 tokens/s

The gap we would fill. These results come from high-end Macs and gaming desktops, and none is a product. We found no one packaging the technique for a 16 GB developer laptop inside an IDE.

All figures are the authors' own measurements, not independently reproduced. Sources: Flash-MoE README; arXiv 2609.18110; lmsys.org blog 2026-08-29; llama.cpp PR #25294, discussion #23324; arXiv 2504.08791.

STANDING ON THEIR SHOULDERS

Their experiments give us a playbook: seven things to do, five to avoid

DO · MEASURED GAINS

Choose an MoE model with a small per-token footprint	**the main lever**
Read experts with parallel direct reads, not memory-mapping	**2.4x to 5x**
Store each expert as one contiguous block on disk	**2.6x to 4.1x**
Let hot experts stay in RAM with a simple cache	**50 to 80% fewer reads**
Pin experts from a sample of real task text	**6% to 42% hits**
Keep experts at 4-bit precision	**tool calling intact**
Drop experts the task never uses	**half removed, same code score**

AVOID · MEASURED LOSSES

Predicting and prefetching the next experts	**18% slower**
Compressing experts on a laptop	**13% slower**
2-bit weights	**breaks tool calls**
Pinning more RAM than is truly free	**0.75 to 0.12 tokens/s**
Dense models from disk	**10 s per token**

Our first report leaned on predicting experts ahead of time. The measured results say caching matters far more than prediction, so the design now puts the cache first.

Sources: Flash-MoE README (58 logged experiments); SSD-LLaMA paper; llama.cpp PR #25294, discussions #23324 and #27149; arXiv 2607.16721. Gains are each project's own figures on its own hardware.

WHAT WE WOULD BUILD

A two-lane assistant: a small model for speed, a big model on SSD for depth

VS Code extension
Chat panel, inline actions, and a job list that shows which model is answering and how long it will take.

requests ↓ ↑ streamed answers, over localhost

LOCAL DAEMON · BACKGROUND SERVICE ON THE LAPTOP

Router
Sends routine requests to the fast lane and hard ones to the deep lane. The user can override.

Memory manager
Holds a hard RAM cap, swaps the lanes, never lets the system swap to disk.

Fast lane
A small model held fully in RAM. Completions and everyday questions, answered in seconds.

On 8 GB laptops this lane can itself be a streamed 30B-class model.

Deep lane · large MoE model
Runs as a background job for reviews, refactoring plans and hard bugs.

Shared weights. Always in RAM.

Expert cache. Hot experts stay in RAM.

Coding profile. Pinned experts, loaded at job start.

Expert reader. Parallel direct reads from SSD.

↑ loaded once at start

↑ experts read on demand

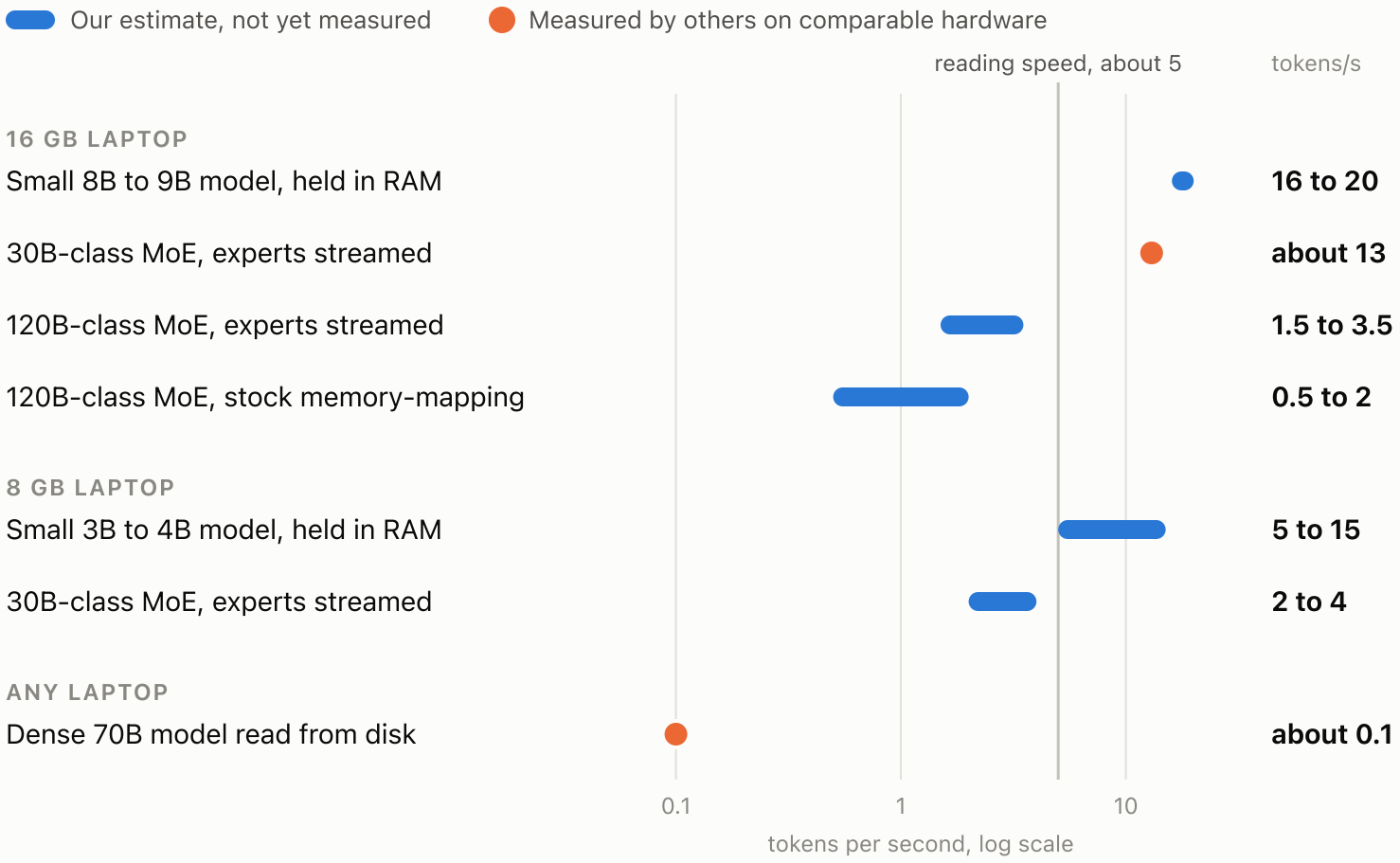
SSD · small model file

SSD · expert pack
The big model, repacked once at install so each expert is a single block that can be read in one request.

Proposed design. The engine underneath is llama.cpp; the expert reader, cache and coding profile are the new work.

EXPECTED PERFORMANCE

On a 16 GB laptop, expect 1.5 to 3.5 tokens per second from a 120B-class model



WHAT THIS MEANS DAY TO DAY

- Routine requests answer in seconds on the small model.
- A hard question sent to the 120B model takes 3 to 6 minutes.
- An 8 GB laptop could run a 30B-class model where today it is limited to 3B or 4B.

The big model is a background reviewer, not a chat partner.

Estimates scale other teams' measurements to our hardware: a 2.9 GB/s SSD against 17.5 GB/s on the Flash-MoE Mac, and a cache of roughly 10% of experts. Phase 0 replaces them with measurements.

Measured points: llama.cpp discussion #23324 (M1 Pro, 16 GB); prima.cpp paper (dense 70B on stock llama.cpp). Small-model range: ModelFit 16 GB guide. All other ranges are our estimates.

Six technical constraints, each with a fix someone has already measured

CONSTRAINT	WHAT GOES WRONG	HOW WE TACKLE IT
SSD speed sets the ceiling	Every expert not in RAM is a disk read. Our SSD reads 2.9 GB/s; the Flash-MoE Mac reads 17.5 GB/s.	Cut the bytes each token needs: MoE model, 4-bit experts, cache, coding profile.
Little free RAM	About 10 GB free on a 16 GB laptop, 3 to 4 GB on 8 GB. Over-filling is worse than under-filling: one test fell from 0.75 to 0.12 tokens/s.	A hard memory cap in the daemon. A smaller model tier on 8 GB. Unload the small model while a big job runs.
Memory-mapping is slow	Stock engines page experts in through the operating system: 0.38 GB/s measured, against 2.8 GB/s for direct reads on the same PC.	Our own expert reader, issuing parallel direct reads of whole experts. Worth 2.4x to 5x in three projects.
File layout	In a standard model file one expert's data is scattered, so loading it touches up to 36 times more of the file than needed.	Repack once at install into an expert pack: one contiguous block per expert.
Long prompts	A prompt uses nearly every expert, so the first word of an answer can take minutes.	Load each expert once per prompt pass (5x faster prompt reading in the llama.cpp PR). Keep big-model context short. Reuse saved prompt state.
Cold start	The first tokens are slow until the cache fills, and a change of topic causes a dip.	Preload the coding profile's pinned experts when a job starts.

Sources: Flash-MoE README; llama.cpp discussion #23324 (Windows read rates, over-pinning test) and #27149 (file layout); llama.cpp PR #25294 (prompt pass); SSD-LLaMA paper.

Seven product constraints, and the design choice that answers each

CONSTRAINT	WHAT GOES WRONG	HOW WE TACKLE IT
Only MoE models qualify	Dense models need all their weights for every token: about 10 seconds per token from disk.	Use MoE models only. Keep the engine model-agnostic and review the model list each quarter.
Answer quality	2-bit weights broke tool calling. Pruned models lose non-coding skills. Some models spread work evenly across experts, which weakens caching.	4-bit minimum. Test every model and profile on our own coding tasks before switching it on.
Not chat speed	A few tokens per second means minutes per answer.	Two lanes. The big model runs as a background job with a visible time estimate.
Heat, battery, energy	Sustained SSD and processor load. Prior work reports up to 12 times more energy per token than in-memory inference.	Run big jobs on mains power by default and pause under thermal pressure.
SSD health	Reading does not wear a drive the way writing does, but swapping would write heavily.	The memory cap prevents swapping. The cache cuts reads. The daemon monitors drive health.
Three operating systems	Fast direct reads use a different interface on macOS, Windows and Linux.	A thin I/O layer per system. macOS first, Windows second. Working examples exist for both.
A moving open-source base	The llama.cpp streaming work is still an open pull request. Flash-MoE ships no licence.	Build on llama.cpp, track that pull request, and reuse published ideas, not unlicensed code.

Sources: Flash-MoE README and InsiderLLM review (2-bit, licence); arXiv 2607.16721 (pruning); SSD-LLaMA paper (energy, citing prior work); llama.cpp discussion #23324 (expert spread, Windows) and PR #25294.

BEING STRAIGHT ABOUT IT

Four things we do not know yet, and one thing this is not

1. **Speed on our own hardware.**
   We found no published result for a 120B-class model on a 16 GB laptop. Every figure for our machines is an estimate until Phase 0, which takes a day.

2. **Whether gpt-oss-120b reuses experts on coding work.**
   The cache depends on it. At least one model family is reported to spread its work more evenly.

3. **Whether the answers justify the wait.**
   The big model must be clearly better than the small one on our own code to be worth minutes.

4. **The effort for the expert reader and cache.**
   It is real engine work. Others have done it, which lowers the risk, but it may run past plan.

WHAT THIS IS NOT

A replacement for a cloud model where cloud is allowed. Cloud is faster and stronger.

WHAT IT IS FOR

- Code that must not leave the laptop
- Work without a network connection
- Zero cost per request on hardware we already own

Expert-spread observation: llama.cpp discussion #23324, about DeepSeek V4.

PLAN AND DECISION

Three phases, two gates, and a first step that takes one day

PHASE 0

Measure

About 1 day

Run stock llama.cpp and the open streaming pull request on a 16 GB Mac and an 8 GB laptop. Record speed, wait before the first word, and memory pressure.

PHASE 1

Two-lane MVP

About 1 week

Daemon, router, memory manager and VS Code extension on stock llama.cpp. Record which experts our coding prompts use.

PHASE 2

Expert store

3 to 6 weeks, less if the open llama.cpp work can be reused

Expert pack, parallel direct reads, cache and coding profile. Re-run the Phase 0 measurements to show the gain.

Gate 1. The 120B-class model reaches 1 token per second on the 16 GB laptop. If not, the project continues with the 30B-class tier only, or stops.

Gate 2. The recorded expert usage shows that a cache and coding profile would at least double speed. If not, the MVP ships as it is.

THE ASK

Approve Phase 0 now. Phase 1 starts only if Gate 1 passes, and Phase 2 only if Gate 2 passes.

WHAT IT NEEDS

- One 8 GB Windows laptop on loan
- About 100 GB of disk on a test Mac
- No purchases and no licence fees

Durations are estimates for AI-assisted development and exclude time for wider team rollout (installer, Windows packaging, updates), estimated at 2 to 3 further weeks.

APPENDIX

Sources

Flash-MoE (open-source project)
github.com/danveloper/flash-moe

SGLang SSD Expert Pack (LMSYS blog)
lmsys.org/blog/2026-08-29-sglang-ssd-expert-pack

llama.cpp discussion #23324: MoE expert offload to disk
github.com/ggml-org/llama.cpp/discussions/23324

Half the Experts, All the Code (domain pruning for coding)
arxiv.org/abs/2607.16721

LLM in a Flash (Apple)
arxiv.org/abs/2312.11514

Prima.cpp (dense 70B baseline on stock llama.cpp)
arxiv.org/abs/2504.08791

OpenAI gpt-oss-120b model card and config
huggingface.co/openai/gpt-oss-120b

M4 MacBook Air SSD benchmark (Gigazine)
gigazine.net/gsc_news/en/20250317-apple-m4-macbook-air-benchmark

SSD-LLaMA: SSD-Native Inference for Trillion-Parameter MoE
arxiv.org/abs/2609.18110

llama.cpp PR #25294: stream MoE routed experts from disk
github.com/ggml-org/llama.cpp/pull/25294

llama.cpp discussion #27149: expert-aware SSD streaming
github.com/ggml-org/llama.cpp/discussions/27149

FlashMoE: ML-based cache replacement for MoE on edge devices
arxiv.org/abs/2601.17063

PowerInfer-2: Fast LLM Inference on a Smartphone
arxiv.org/abs/2406.06282

HOBBIT: mixed-precision expert offloading
arxiv.org/abs/2411.01433

Flash-MoE review: what a 397B model on a laptop cost
insiderllm.com/guides/flash-moe-run-397b-model-laptop

MacBook Air M4 16 GB guide (ModelFit)
modelfit.io/blog/best-llm-macbook-air-m4-16gb

Every page above was opened and read on 11 to 12 November 2026. Figures quoted from them are the authors' own and have not been independently reproduced. Estimates for our hardware are ours and are labelled as such on each slide.